

Aluno: Nicolas Araújo Fonseca Pimenta

Resumo Artigo:

On the Criteria To Be Used in Decomposing Systems into Modules

O artigo discute a modularização como um mecanismo para melhorar a flexibilidade e a compreensibilidade de um sistema, permitindo a redução do seu tempo de desenvolvimento. O autor postula que a eficácia de uma estratégia de divisão de software é estritamente dependente dos critérios utilizados para segmentar o sistema em módulos. Na visão de **Parnas**, a definição de "módulo" transcende a mera criação de subprogramas, sendo interpretada como uma atribuição de responsabilidades que inclui decisões de design tomadas antes do início do trabalho independente.

Para fundamentar sua tese, o autor utiliza o **Example System 1: A KWIC Index Production System**. Este sistema processa um conjunto de linhas, gerando todas as suas permutações circulares e listando-as em ordem alfabética. Embora seja um problema de pequena escala, ele serve como modelo para demonstrar como grandes projetos podem ser estruturados de formas radicalmente distintas, resultando em diferentes níveis de manutenção e adaptabilidade.

1. A Falácia da Decomposição Baseada no Fluxograma (Modularization 1)

A primeira abordagem descrita, denominada **Modularization 1**, reflete o método convencional de desenvolvimento, onde o sistema é dividido de acordo com as etapas sequenciais de processamento de dados. Nesta estrutura, os módulos correspondem a fases lógicas: **Input**, **Circular Shift**, **Alphabetizing**, **Output** e **Master Control**.

O problema central desta abordagem é que ela se assemelha a um **flowchart**. Como resultado, as decisões de design sobre formatos de dados, como o empacotamento de caracteres em palavras ou a estrutura de tabelas de índices, tornam-se interfaces complexas compartilhadas entre os módulos. Isso cria uma dependência sistêmica onde uma alteração em um detalhe técnico exige modificações em todos os componentes, comprometendo a **product flexibility**.

2. O Princípio de Information Hiding (Modularization 2)

Em oposição ao modelo tradicional, a **Modularization 2** é construída sob o critério de **Information Hiding**. Nesta filosofia, cada módulo é caracterizado pelo conhecimento de uma decisão de design específica que ele "esconde" de todos os outros. Os módulos funcionam como provedores de serviços abstratos; por exemplo, o módulo de **Line Storage** oferece funções como `$CHAR(r,w,c)$`, ocultando como os caracteres estão fisicamente organizados.

Essa abordagem garante que as interfaces sejam mais abstratas, baseando-se em nomes de funções e parâmetros em vez de formatos de tabelas. Consequentemente, o desenvolvimento independente pode começar mais cedo, e a **comprehensibility** do sistema é aumentada, já

que um desenvolvedor pode entender a lógica de um módulo sem precisar desvendar as complexidades internas de outros componentes.

3. Eficiência, Implementação e a Independência de Representação

Um dos pontos mais sofisticados da argumentação de **Parnas** refere-se à **Efficiency and Implementation**. O autor reconhece que a **Modularization 2**, se implementada de forma ingênua com chamadas de sub-rotinas convencionais e sequências de chamadas elaboradas, pode resultar em uma perda de desempenho devido ao excesso de alternância entre módulos. Em contraste, a **Modularization 1** é naturalmente eficiente nesse aspecto, pois as transferências de controle são infrequentes.

Para mitigar esse problema sem sacrificar os benefícios do design, o autor propõe que abandonemos a premissa de que um módulo deve corresponder estritamente a um conjunto de sub-rotinas em tempo de execução. Ele sugere o uso de ferramentas, como assemblers ou técnicas de mapeamento, que permitam ao programador visualizar e alterar o sistema de forma modular, enquanto a máquina processa uma representação altamente integrada e eficiente. Assim, o software pode possuir múltiplas representações: uma para a compreensão humana e manutenção, e outra, possivelmente distinta e otimizada, para a execução. Essa distinção é crucial para que a modularidade não seja descartada em nome de uma eficiência que pode ser alcançada por meios técnicos de tradução de código.

4. Estrutura Hierárquica e a Independência de Níveis

Para finalizar a análise, **Parnas** introduz a relação entre a modularização e a **Hierarchical Structure**. Ele define que um sistema possui uma hierarquia quando existe uma relação de dependência parcial, como "usa" ou "depende de", entre seus componentes. O autor demonstra que a **Modularization 2** permite a criação de uma hierarquia clara, onde módulos de baixo nível, como o **Line Storage**, podem funcionar independentemente de módulos de alto nível, como o **Alphabetizer**.

A principal vantagem dessa organização hierárquica é a possibilidade de "podar" (**prune**) as camadas superiores para reutilizar a base do sistema em novos projetos. Por exemplo, o módulo de armazenamento de linhas poderia ser a base de um sistema de perguntas e respostas sem a necessidade de carregar as funções de indexação KWIC. O autor conclui que, embora a hierarquia e a decomposição por ocultação de informação sejam propriedades independentes, a combinação de ambas resulta em sistemas que são simultaneamente fáceis de entender, flexíveis para mudar e robustos para reutilização.